

URL Shortener System Design

URL Shortener System Design

Let's design a URL shortener like bit.ly. This is a solid system design problem that covers most of the important distributed systems concepts you would encounter in real projects.

1 Step 1: Understanding Requirements

1.1 What we need to build

- Take long URLs and make them short
- Redirect users when they click short URLs
- Let users pick custom short links (when available)
- Set expiration dates on URLs
- Show basic analytics like click counts and geographic data
- Handle user accounts (optional but nice to have)

1.2 Scale and performance targets

- Support 100 million daily active users
- Handle 100 million URL shortenings per day
- Process 10 billion redirects daily (that's roughly 100:1 read to write ratio)
- Keep URLs around for 5 years
- Redirects should be fast - under 100ms
- Stay up 99.9% of the time
- Handle traffic spikes gracefully

1.3 Quick math on scale

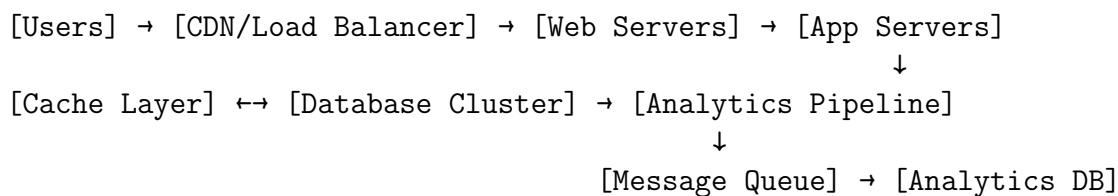
Breaking down the numbers:

- Write QPS: $100\text{M} / (24 \times 3600) \approx 1,200/\text{second}$
- Read QPS: $10\text{B} / (24 \times 3600) \approx 115,000/\text{second}$
- Total URLs over 5 years: $100\text{M} \times 365 \times 5 = 180$ billion URLs
- Storage needed: $180\text{B} \times 500 \text{ bytes} \approx 90\text{TB}$ (assuming 500 chars per URL on average)
- Bandwidth: $115\text{K QPS} \times 500 \text{ bytes} \approx 55 \text{ MB/s}$ outbound

This is definitely "big scale" territory, so we need to consider sharding, caching, CDNs, and other scalability measures.

2 Step 2: High-level Architecture

The basic system flow is as follows:



2.1 API design

We keep the API simple but complete:

Shorten a URL:

POST /api/v1/shorten

Content-Type: application/json

```
{
  "url": "https://example.com/some/really/long/path/with/many/parameters",
  "custom": "my-custom-link",      // optional
  "expires": "2024-12-31T23:59:59Z", // optional
  "user_id": 12345                  // optional
}
```

Response:

```
{
  "short_url": "https://s.ly/abc123",
  "original_url": "https://example.com/...",
}
```

```

    "created_at": "2024-01-01T00:00:00Z",
    "expires_at": "2024-12-31T23:59:59Z"
}

```

Redirect:

```

GET /abc123
→ HTTP 302 Found
→ Location: https://example.com/original/url

```

Analytics:

```

GET /api/v1/analytics/abc123
Response:
{
  "total_clicks": 15420,
  "daily_clicks": [{"date": "2024-01-01", "clicks": 150}, ...],
  "countries": {"US": 45, "UK": 20, "CA": 15},
  "referrers": {"google.com": 30, "twitter.com": 25}
}

```

3 Step 3: Detailed Design

3.1 URL encoding strategy

The heart of the system is generating short codes. We use base62 encoding (a-z, A-Z, 0-9) because:

- 7 characters gives us $62^7 = 3.5 \text{ trillion possible URLs}$ This is far more than our 180B requirement
- Looks clean and readable
- No confusing characters like 0/O or 1/I

Instead of hashing (which can have collisions), we use a counter approach:

```

def encode\textbackslash{}_base62(num):
    chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
    if num == 0:
        return chars[0]

    result = ""
    while num > 0:
        result = chars[num \% 62] + result
        num = num // 62

```

```
    num //= 62
    return result.rjust(7, chars[0]) \textbackslash{}# pad to 7 chars
```

For the counter service, we use a distributed approach:

- Multiple counter servers, each handling a range (server1: 1-1M, server2: 1M-2M, etc.)
- When a server runs out, it requests the next range from the coordinator
- No single point of failure

3.2 Database design

We use MySQL with sharding because we need ACID properties for URL creation, and it handles our scale requirements well.

Main URLs table:

```
CREATE TABLE urls (
    id BIGINT PRIMARY KEY,
    short_code VARCHAR(10) UNIQUE NOT NULL,
    original_url TEXT NOT NULL,
    user_id BIGINT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NULL,
    click_count BIGINT DEFAULT 0,
    is_active BOOLEAN DEFAULT TRUE,

    INDEX idx_short_code (short_code),
    INDEX idx_user_id (user_id),
    INDEX idx_created_at (created_at),
    INDEX idx_expires_at (expires_at)
);
```

Users table (if we support accounts):

```
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    email VARCHAR(255) UNIQUE,
    password_hash VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    plan_type ENUM('free', 'premium') DEFAULT 'free'
);
```

Sharding strategy:

- Shard by `hash(short_code) % num_shards`
- Start with 64 shards (can handle 180B URLs easily)
- Use consistent hashing for easier resharding later
- Each shard can be a master-slave pair for read scaling

3.3 Caching layers

This is critical for achieving our 95%+ cache hit rate target:

Browser cache:

- Cache 302 redirects for 1 hour
- Reduces server load significantly

CDN layer (CloudFlare/AWS CloudFront):

- Cache popular URLs at edge locations worldwide
- Especially useful for viral links
- TTL: 24 hours for active URLs

Application cache (Redis cluster):

```
\section*{Hot data cache}
```

```
hot_urls_cache:
```

```
  ttl: 3600  # 1 hour
  max_memory: 20GB
  eviction: allkeys-lru
```

```
\section*{Analytics cache}
```

```
analytics_cache:
```

```
  ttl: 300  # 5 minutes
  max_memory: 5GB
```

```
\section*{Rate limiting cache}
```

```
rate_limit_cache:
```

```
  ttl: 3600
  max_memory: 2GB
```

Cache warming strategy: preload the top 1% of URLs based on historical data.

3.4 Rate limiting

We cannot let people spam our service. We use the token bucket algorithm:

```
class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate # tokens per second
        self.last_refill = time.time()

    def consume(self, tokens=1):
        self._refill()
        if self.tokens >= tokens:
            self.tokens -= tokens
            return True
        return False

    def _refill(self):
        now = time.time()
        tokens_to_add = (now - self.last_refill) * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + tokens_to_add)
        self.last_refill = now
```

Rate limits:

- Anonymous users: 100 URL creations per hour per IP
- Registered users: 1000 per hour
- Premium users: 10000 per hour
- Global circuit breaker at 80% system capacity

3.5 Analytics pipeline

We do not want analytics to slow down redirects, so everything is asynchronous:

```
[Click Event] → [Kafka] → [Stream Processor] → [Analytics DB]
                  ↓
                [Real-time Dashboard]
```

Analytics data model:

```

-- Raw click events (kept for 30 days)
CREATE TABLE click_events (
    id BIGINT PRIMARY KEY,
    short_code VARCHAR(10),
    timestamp TIMESTAMP,
    ip_address VARCHAR(45),
    user_agent TEXT,
    referrer VARCHAR(500),
    country_code CHAR(2),

    INDEX idx_short_code_time (short_code, timestamp)
);

-- Daily aggregated stats (kept forever)
CREATE TABLE daily_analytics (
    short_code VARCHAR(10),
    date DATE,
    total_clicks INT,
    unique_ips INT,
    top_countries JSON,
    top_referrers JSON,

    PRIMARY KEY (short_code, date)
);

```

Stream processing with Kafka Streams includes:

- Real-time aggregation of click data
- Geographic lookup using IP
- Bot filtering
- Anomaly detection

3.6 Security and validation

URL validation:

```

def validate_url(url):
    # Basic format check
    if not url.startswith(('http://', 'https://')):
        return False

    # Length check
    if len(url) > 2048:

```

```

    return False

# Malware/phishing check against blacklists
if is_malicious_url(url):
    return False

# No self-references
if 's.ly' in url:
    return False

return True

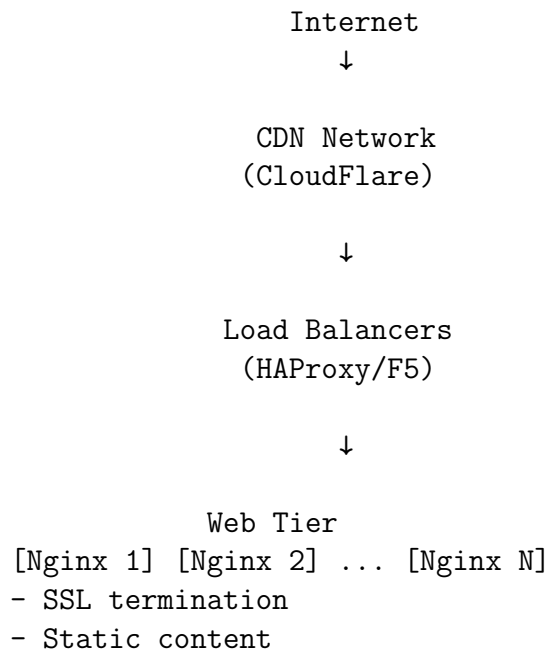
```

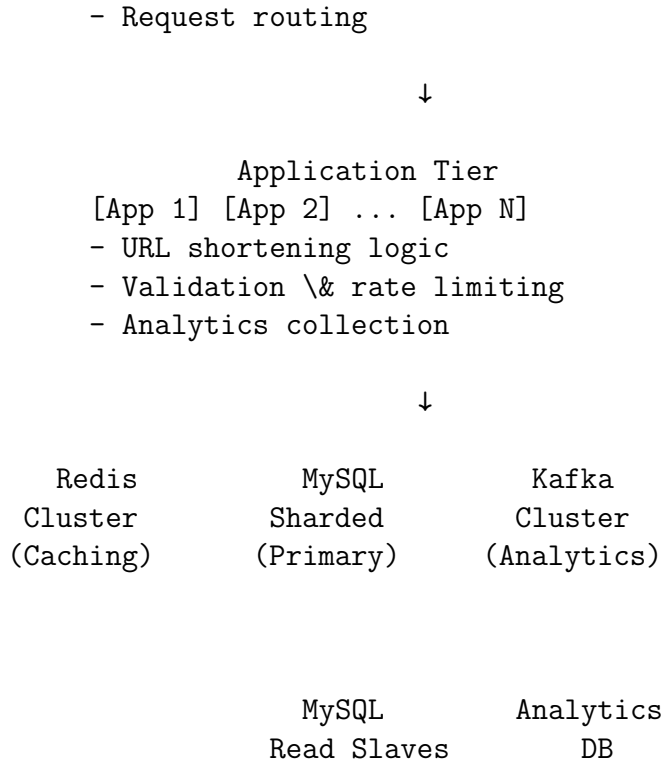
Other security measures:

- HTTPS everywhere
- SQL injection prevention (parameterized queries)
- XSS protection in analytics dashboard
- Rate limiting to prevent DDoS
- Regular security audits

4 Step 4: Putting it all together

4.1 Complete system architecture





4.2 Detailed user flows

Creating a short URL: 1. User submits long URL via web/API 2. Web server validates request format 3. Rate limiter checks if user can create more URLs 4. App server validates URL (format, length, blacklist check) 5. Generate unique ID from counter service 6. Convert ID to base62 short code 7. Check if custom alias requested and available 8. Store mapping in database (with retry logic) 9. Pre-warm Redis cache with new mapping 10. Log creation event for analytics 11. Return short URL to user

Clicking a short URL (the hot path): 1. User clicks short link (e.g., s.ly/abc123) 2. CDN checks if it has the redirect cached - If yes: return cached redirect (fastest path) 3. Load balancer routes to web server 4. Web server extracts short code 5. Redis cache lookup for short code - Cache hit: get original URL immediately - Cache miss: query MySQL database 6. If database query needed: - Check read replica first - Fallback to master if needed - Update Redis cache for future requests 7. Log click event to Kafka (async, no blocking) 8. Return 302 redirect to browser 9. Browser redirects user to original URL

Background analytics processing:

- Kafka consumer processes click events
- Geographic lookup, bot detection
- Real-time aggregation
- Update daily/hourly stats tables

4.3 Monitoring and operations

Key metrics to watch:

- **Latency:** P50, P95, P99 redirect times (target: P95 ; 100ms)
- **Availability:** Uptime percentage (target: 99.9%)
- **Cache hit rate:** Redis hit rate (target: ;95%)
- **Error rate:** 4xx/5xx response percentage (target: ;0.1%)
- **Database:** Connection pool usage, query times, replication lag
- **Queue lag:** Kafka consumer lag for analytics
- **Rate limiting:** Requests blocked per minute

Alerting setup:

alerts:

- name: HighLatency
condition: p95_redirect_latency > 100ms
duration: 2m
severity: warning
- name: CacheHitRateLow
condition: redis_hit_rate < 90\%
duration: 5m
severity: warning
- name: DatabaseDown
condition: mysql_master_up == false
severity: critical
- name: QueueLag
condition: kafka_consumer_lag > 1000
duration: 5m
severity: warning

Operational procedures:

- Blue-green deployments for zero downtime
- Automated failover for database masters
- Circuit breakers to prevent cascade failures
- Graceful degradation (disable analytics if needed)
- Regular backup testing and disaster recovery drills

5 Key design decisions and trade-offs

Why base62 counter over hash-based?

- Hash-based: Can have collisions, need ugly collision resolution, variable length output
- Counter-based: Guaranteed unique, predictable length, clean implementation
- Trade-off: Need distributed counter service, but it's simpler than collision handling

Why MySQL over NoSQL?

- Need strong consistency for URL creation (can't have duplicate short codes)
- Complex queries for analytics would be painful in NoSQL
- MySQL with sharding handles our scale just fine
- Trade-off: Slightly more complex sharding vs eventual consistency issues

Why async analytics?

- Redirects need to be under 100ms - can't afford database writes
- Analytics can tolerate some delay (few seconds is fine)
- Decoupling makes system more reliable
- Trade-off: Slightly delayed analytics vs much faster redirects

Cache vs database for redirects?

- Cache-first approach crucial for performance
- Database as fallback ensures reliability
- 95%+ hit rate makes database load manageable
- Trade-off: More complexity vs much better performance

6 Scaling and future improvements

Immediate scaling strategies:

- Add more read replicas for database
- Increase Redis cluster size
- Add more application servers (they're stateless)
- Geographic distribution (multiple regions)

Future enhancements:

- **Machine learning:** Predict URL popularity for cache preloading
- **Advanced analytics:** User behavior analysis, conversion tracking
- **A/B testing:** Different redirect strategies
- **API gateway:** Centralized API management and authentication
- **Microservices:** Break URL service into smaller components
- **Real-time dashboard:** Live analytics for premium users

Potential bottlenecks and solutions:

- Counter service: Shard counter ranges across multiple servers
- Database writes: Use write-through caching, batch operations
- Analytics processing: Add more Kafka partitions and consumers
- Geographic latency: Deploy to more regions

This design handles our scale requirements (115K read QPS, 1.2K write QPS) while maintaining sub-100ms latency and 99.9% availability. The key is aggressive caching, asynchronous processing where possible, and keeping the redirect path as simple as possible.